

C# Programming Concepts

1. Variables

What is a Variable?

A **variable** is a named memory location used to store data. The value stored in a variable can change during program execution.

Syntax

```
dataType variableName = value;
```

Example

```
int age = 25;
```

```
string name = "John";
```

```
double salary = 45000.50;
```

Rules for Variable Names

- Must start with a letter or underscore (_)
- Cannot start with a number
- Cannot contain spaces
- Cannot use C# keywords
- Variable names are case-sensitive

Example:

```
int Age = 20;
```

```
int age = 25; // Different variable
```

Best Practices

- Use meaningful names
- Follow camelCase naming convention

```
int studentAge;
```

```
string employeeName;
```

```
string employeeOfficeAddress;
```

```
double employeeSalary;
```

2. Data Types

What is a Data Type?

A **data type** specifies the type of data that a variable can store.

Categories

Value Types

Stored directly in memory.

Data Type	Size	Example
int	4 bytes	10
long	8 bytes	123456789
short	2 bytes	100
byte	1 byte	255
float	4 bytes	12.5f
double	8 bytes	12.567
decimal	16 bytes	100.25m
char	2 bytes	'A'
bool	1 byte	true

Example

```
int marks = 90;
```

```
bool passed = true;
```

```
char grade = 'A';
```

Reference Types

Store the memory address of the object.

Examples

```
string name = "Ravi";
```

```
object obj = new object();
```

Common Reference Types

- string
- object
- arrays
- classes
- interfaces
- delegates

var Keyword

Automatically determines the data type.

```
var age = 25;
```

```
var name = "John";
```

Compiler converts them to

```
int age = 25;
```

```
string name = "John";
```

3. Type Conversion

Changing one data type into another.

Implicit Conversion

Automatic conversion (small → large)

```
int num = 100;
```

```
double d = num;
```

Explicit Conversion (Casting)

Manual conversion

```
double d = 100.75;
```

```
int num = (int)d;
```

Output

100

Convert Class

```
string age = "25";
int a = Convert.ToInt32(age);
```

Parse()

```
string num = "200";
int n = int.Parse(num);
```

TryParse()

Safely converts without exceptions.

```
string input = "100";

bool result = int.TryParse(input, out int number);
Console.WriteLine(number);
```

4. Operators

Operators perform operations on variables.

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example

```
int a = 20;
int b = 10;
```

Relational Operators

Used for comparison.

Operator	Meaning
==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater or Equal
<=	Less or Equal

Example

```
Console.WriteLine(10 > 5);
```

Output

True

Logical Operators

Operator	Meaning
&&	AND
!	NOT
	OR

Example

```
bool result = (10 > 5) && (20 > 15);
```

Assignment Operators : = += -= *= /=

Example

```
int x = 10;
```

```
x += 5;
```

Result

15

Increment / Decrement

```
x++;
```

```
x--;
```

5. If-Else Statement

Used for decision making.

if Statement

```
int age = 20;
```

```
if(age >= 18)
```

```
{
```

```
    Console.WriteLine("Eligible");
```

```
}
```

if-else

```
if(age >= 18)
```

```
{
```

```
    Console.WriteLine("Adult");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Minor");
```

```
}
```

else if Ladder

```
int marks = 75;
```

```
if(marks >= 90)
```

```
{
```

```
    Console.WriteLine("Grade A");
```

```
}
```

```
else if(marks >= 75)
```

```
{
```

```
    Console.WriteLine("Grade B");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Grade C");
```

```
}
```

Nested if

```
if(age >=18)
```

```
{
```

```
    if(age <=60)
```

```
    {
```

```
        Console.WriteLine("Working Age");
```

```
    }
```

```
}
```

6. Switch Statement

Used when multiple choices exist.

Example

```
int day = 2;
```

```
switch(day)
{
    case 1:
        Console.WriteLine("Monday");
        break;

    case 2:
        Console.WriteLine("Tuesday");
        break;

    case 3:
        Console.WriteLine("Wednesday");
        break;

    default:
        Console.WriteLine("Invalid");
        break;
}
```

7. Loops

Loops repeat a block of code.

for Loop

Best when number of iterations is known.

```
for(int i=1;i<=5;i++)
{
    Console.WriteLine(i);
}
```

while Loop

Runs while condition is true.

```
int i=1;
```

```
while(i<=5)
{
    Console.WriteLine(i);
    i++;
}
```

do-while Loop

Executes at least once.

```
int i=1;
```

```
do
{
    Console.WriteLine(i);
    i++;
}
while(i<=5);
```

foreach Loop

Used to iterate collections.

```
string[] names = { "John", "Ravi", "David" };
```

```
foreach(string name in names)
{
    Console.WriteLine(name);
}
```

8. Arrays

What is an Array?

An **array** stores multiple values of the same data type using a single variable.

Declaration

```
int[] numbers = new int[5];
```

Initialization

```
int[] numbers = { 10, 20, 30, 40, 50 };
```

Accessing Elements

```
Console.WriteLine(numbers[0]);
```

```
Console.WriteLine(numbers[2]);
```

Output

10

30

Modifying Elements

```
numbers[1] = 200;
```

Array Length

```
Console.WriteLine(numbers.Length);
```

Output

5

Loop Through Array

```
for(int i=0;i<numbers.Length;i++)
```

```
{
```

```
    Console.WriteLine(numbers[i]);
```

```
}
```

foreach with Array

```
foreach(int n in numbers)
```

```
{
    Console.WriteLine(n);
}
```

Summary

Topic	Purpose
Variables	Store values in memory
Data Types	Define the type of data a variable can hold
Operators	Perform arithmetic, comparison, logical, and assignment operations
Type Conversion	Convert data from one type to another
Nullable Types	Allow value types to hold null values
If-Else	Execute code based on conditions
Switch	Select one block of code from multiple options
Loops	Repeat code efficiently (for, while, do-while, foreach)
Arrays	Store and manage multiple values of the same type